

UNIVERSIDAD PRIVADA DEL NORTE

Facultad de Ingeniería de sistemas Computacionales

Sistema de Inventariado para Almacén

Presentado por:

- Caceres Cotaquispe Keber Royer - N00224396
- Lima Jiménez Waldir - N00431592
- Reginaldo Paucar Ángel Eduardo - N00458156
- Meléndez Silva Jhojan Francis - N00311207
- Candro Juli Juan - N00486838

Docente: Torres Rodríguez Martin Eduardo



Lima, 2026

1. Definición del Problema

1.1 Situación Problemática

Actualmente, muchas empresas con almacenes medianos y pequeños gestionan sus inventarios de forma manual o mediante hojas de cálculo básicas. Esto genera pérdidas de tiempo, errores humanos frecuentes, dificultad para conocer el stock real en tiempo real y una toma de decisiones deficiente por falta de información oportuna y confiable.

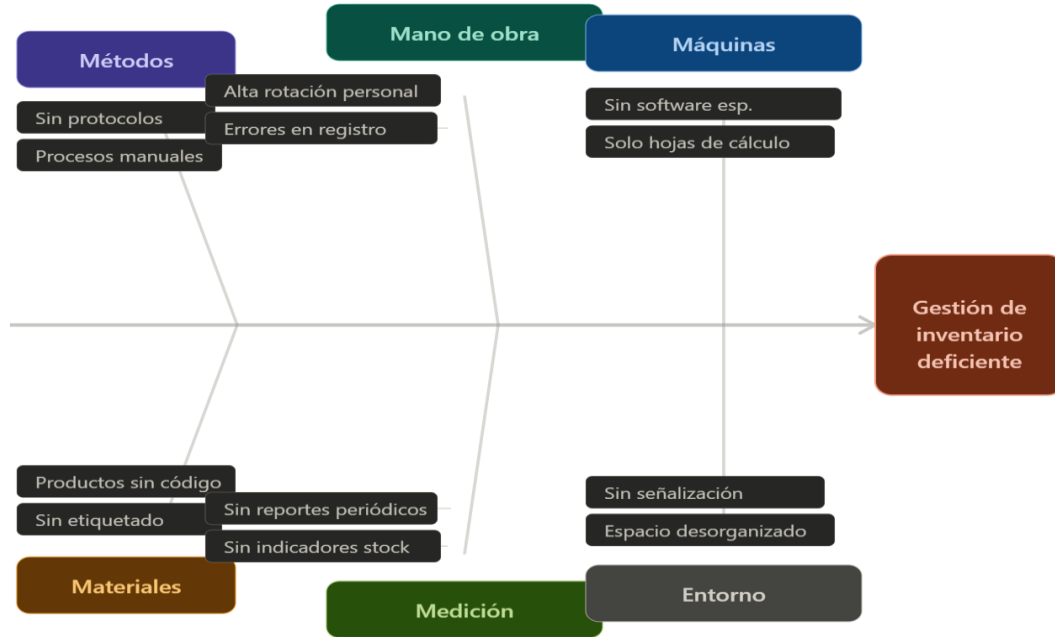
La ausencia de un sistema centralizado provoca que los encargados de almacén no puedan identificar con rapidez qué productos están agotándose, cuáles tienen exceso de stock o cuáles han superado su fecha de vencimiento. Del mismo modo, la comunicación entre el área de compras, ventas y almacén se ve entorpecida al no contar con una fuente de datos unificada es por que nos basamos en las principales problemáticas de la Ferretería Santa Cruz S.R.L. se centran en deficiencias logísticas y operativas, específicamente en la gestión de almacenes e inventarios, lo que provoca demoras en las entregas. Los problemas operativos y administrativos detectados en la empresa incluyen: Desorganización en el almacén: Falta de orden y limpieza. Los productos no están correctamente clasificados, lo que dificulta el despacho rápido. Falta de control de existencias: Ausencia de fichas de control y de órdenes de compra estandarizadas. Desabastecimiento: La empresa enfrenta frecuentemente problemas con escasez de stock para responder a la demanda.

1.2 Problema Identificado

La empresa no cuenta con un sistema de software que permita registrar, controlar y hacer seguimiento de los productos en su almacén de manera eficiente, lo que ocasiona desabastecimiento, sobrestockeo, pérdidas económicas y una gestión operativa ineficiente.

1.3 Diagrama de Ishikawa

A continuación, se identifican las principales causas raíz del problema, organizadas bajo las categorías clásicas del Diagrama de Causa-Efecto (Ishikawa):



2. Antecedentes

Para el desarrollo del presente proyecto se revisaron investigaciones previas relacionadas con sistemas de gestión de inventarios y el uso de la Programación Orientada a Objetos en soluciones empresariales.

IPN México (2010)

Desarrollaron un sistema de inventario en Java usando POO, destacando que el encapsulamiento facilita el mantenimiento del código y que los cambios en los datos se propagan automáticamente. Ese enfoque es el mismo que aplicamos aquí.

Instituto Politécnico Nacional. (2010). Sistema de inventario OO en Java [Tesina]. CIATEQ. <https://tesis.ipn.mx/bitstream/handle/123456789/5757/C7.1410.pdf>

Universidad Continental — Perú (2023)

Implementaron un sistema de gestión de inventarios con alertas automáticas de stock mínimo, funcionalidad que también incluimos en nuestro proyecto. Nos ayudó a definir el alcance funcional.

Roman Arce, J. (2023). Implementación de un sistema de gestión de inventarios [Tesis].
Universidad Continental.

https://repositorio.continental.edu.pe/bitstream/20.500.12394/13413/2/IV_FIN_108_TE_Roman_Arce_2023.pdf

Universidad Tecnológica del Perú (2022)

Proyecto universitario de POO enfocado en ventas e inventario usando Java SE y archivos de texto, sin base de datos. Contexto muy similar al nuestro y confirmó que es viable con los recursos de un curso.

Barreto, L., Guillen, M., Gomez, J. y Damian, E. (2022). Sistema de gestión de ventas e inventario [Trabajo de curso]. UTP.

Laudon & Laudon (2016)

Libro de referencia que explica por qué automatizar procesos manuales reduce errores y mejora la toma de decisiones en las empresas. Sirve como marco teórico del proyecto.

Laudon, K. C. y Laudon, J. P. (2016). Sistemas de información gerencial (14.^a ed.). Pearson.

3. Restricciones Realistas

3.1 Restricciones Técnicas

- Java SE sin frameworks externos, como lo exige el curso.
- Persistencia en archivos .txt o .csv, sin base de datos.
- Interfaz de consola (CLI); Swing como opción adicional si el tiempo lo permite.
- Windows 10/11 con JDK 11 o superior.
- Sin conectividad a internet ni servicios externos.

3.2 Restricciones Económicas

- Solo herramientas gratuitas: JDK, IntelliJ Community o VS Code.
- Sin pagos de nube ni licencias externas.
- Disponibilidad de ~6 horas semanales por integrante.

3.3 Restricciones Operativas

- Catálogo máximo de 500 productos.
- Sistema monousuario, sin accesos simultáneos.
- Sin control de roles en esta versión.
- Plazo: ciclo académico actual (~16 semanas).
- Funcionalidades alineadas con los temas del curso: herencia, polimorfismo, colecciones y archivos.

4. Objetivos del Proyecto

4.1 Objetivo General

Desarrollar un sistema de inventariado para almacén empresarial utilizando el paradigma de Programación Orientada a Objetos en Java, que permita registrar, controlar y gestionar el movimiento de productos (entradas, salidas y stock), con persistencia de datos en archivos, contribuyendo a la eficiencia operativa del proceso de gestión de almacén.

4.2 Objetivos Específicos

- OE1: Modelar el dominio del problema mediante un diagrama de clases UML que aplique correctamente los principios de herencia, encapsulamiento y polimorfismo.
- OE2: Implementar las clases principales del sistema (Producto, Categoría, Almacén, Movimiento, Usuario) con sus atributos, métodos y relaciones correspondientes.
- OE3: Desarrollar las funcionalidades de registro de entradas y salidas de productos, actualización automática de stock y generación de alertas por stock mínimo.
- OE4: Implementar la persistencia de datos mediante el manejo de archivos en Java (FileWriter, BufferedReader, etc.), garantizando la integridad de la información entre sesiones.
- OE5: Crear una interfaz de usuario (consola o Swing) que permita a los operadores del almacén interactuar de forma intuitiva con el sistema.
- OE6: Validar el correcto funcionamiento del sistema mediante pruebas funcionales que verifiquen el cumplimiento de los requerimientos definidos.
- OE7: Documentar el código fuente aplicando buenas prácticas de programación (comentarios Javadoc, nombres descriptivos, separación de responsabilidades).

5. Alcance de la Solución

El sistema de inventariado propuesto abarca la gestión integral del stock de productos en un almacén empresarial, cubriendo desde el registro inicial de productos hasta el seguimiento de movimientos y la generación de reportes básicos. A continuación, se detalla el alcance funcional y técnico del proyecto.

5.1 Alcance Funcional

El sistema contempla las siguientes funcionalidades principales:

Funcionalidades incluidas

- Registrar, editar y eliminar productos con su categoría.
- Registrar entradas y salidas con actualización automática de stock.
- Alertas cuando el stock baje del mínimo configurado.
- Búsqueda de productos por nombre, código o categoría.
- Historial de movimientos por rango de fechas.
- Persistencia total en archivos de texto.

Fuera del alcance

- Integración con ERP o bases de datos externas.
- Múltiples usuarios con roles y permisos.
- Módulo de facturación o pagos.
- Acceso desde red o dispositivos móviles.

5.2 Alcance Técnico

Aspecto	Detalle
Lenguaje	Java SE 11+
Paradigma	POO
Persistencia	Archivos .txt/.csv (java.io)
Interfaz	Consola CLI
Versiones	Git / GitHub
IDE	IntelliJ Community / VS Code
Plataforma	Windows 10/11 con JDK 11+

En resumen, el presente proyecto tiene como alcance entregar un prototipo funcional de escritorio que demuestre la correcta aplicación de los conceptos de POO vistos en el curso, resolviendo una problemática real de gestión de almacén en un entorno académico controlado.

6. Requerimientos Funcionales

A continuación, se detallan los requerimientos funcionales del Sistema de Inventariado para Almacén. Cada requerimiento describe una capacidad específica que el sistema debe ofrecer para cubrir las necesidades operativas de la Ferretera Santa Cruz S.R.L.

Gestión de Productos

- ✓ RF01. Registrar producto.

El sistema debe permitir registrar nuevos productos ingresando código, nombre, descripción, categoría, precio, unidad de medida, stock actual y stock mínimo.

- ✓ RF02. Modificar producto.

El sistema debe permitir actualizar la información de un producto previamente registrado.

- ✓ RF03. Eliminar producto.

El sistema debe permitir eliminar productos que ya no formen parte del inventario.

- ✓ RF04. Buscar producto por código.

El sistema debe permitir localizar un producto mediante su código único.

- ✓ RF05. Buscar producto por nombre.

El sistema debe permitir encontrar productos utilizando búsquedas parciales o completas por nombre.

- ✓ RF06. Mostrar todos los productos.

El sistema debe listar todos los productos registrados en el inventario.

- ✓ RF07. Consultar stock.

El sistema debe mostrar la cantidad disponible de cada producto.

- ✓ RF08. Actualizar stock.

El sistema debe actualizar automáticamente las existencias después de cada movimiento.

- ✓ RF09. Registrar precio.

El sistema debe permitir asignar un precio al momento de registrar un producto.

- ✓ RF10. Modificar precio.

El sistema debe permitir actualizar el precio de un producto cuando sea necesario.

Gestión de Movimientos de Inventario

- ✓ RF11. Registrar entrada de stock.

El sistema debe registrar el ingreso de mercadería al almacén.

- ✓ RF12. Registrar salida de stock.

El sistema debe registrar la salida de productos por venta, devolución o merma.

- ✓ RF13. Validar stock disponible.

El sistema debe verificar que exista suficiente stock antes de realizar una salida.

- ✓ RF14. Registrar motivo del movimiento.

El sistema debe almacenar el motivo de cada entrada o salida de productos.

- ✓ RF15. Registrar fecha del movimiento.

El sistema debe guardar automáticamente la fecha y hora de cada movimiento realizado.

- ✓ RF16. Consultar historial de movimientos.

El sistema debe mostrar el historial de entradas y salidas de un producto.

- ✓ RF17. Buscar movimientos por fecha.

El sistema debe permitir filtrar movimientos según un rango de fechas.

- ✓ RF18. Mostrar movimientos recientes.

El sistema debe listar los últimos movimientos registrados en el almacén.

- ✓ RF19. Registrar stock mínimo.

El sistema debe permitir definir un nivel mínimo de existencias para cada producto.

- ✓ RF20. Generar alerta de stock crítico.

El sistema debe notificar cuando un producto alcance o esté por debajo del stock mínimo.

Reportes

- ✓ RF21. Mostrar productos con bajo stock.

El sistema debe listar los productos cuyo stock sea igual o menor al mínimo establecido.

- ✓ RF22. Mostrar productos agotados.

El sistema debe identificar los productos que no tienen existencias disponibles.

- ✓ RF23. Mostrar entradas registradas.

El sistema debe generar un reporte de ingresos de productos.

- ✓ RF24. Mostrar salidas registradas.

El sistema debe generar un reporte de egresos de productos

- ✓ RF25. Mostrar movimientos por producto.

El sistema debe permitir consultar todos los movimientos asociados a un producto específico.

- ✓ RF26. Mostrar movimientos por categoría.

El sistema debe generar reportes agrupados por categoría.

- ✓ RF27. Mostrar inventario general.

El sistema debe generar un reporte completo del inventario actual.

- ✓ RF28. Mostrar alertas activas.

El sistema debe presentar los productos que requieren reposición inmediata.

- ✓ RF29. Exportar reporte a archivo.

El sistema debe guardar reportes en archivos de texto o CSV.

- ✓ RF30. Mostrar resumen de inventario.

El sistema debe mostrar estadísticas generales del almacén.

Sistema

- ✓ RF31. Iniciar sistema.

El sistema debe permitir el acceso a las funcionalidades del inventario.

- ✓ RF32. Mostrar menú principal.

El sistema debe presentar las opciones disponibles para el usuario.

- ✓ RF33. Validar opciones del menú.

El sistema debe verificar que las opciones ingresadas sean válidas.

- ✓ RF34. Guardar información en archivos.

El sistema debe almacenar los datos en archivos de texto o CSV.

- ✓ RF35. Cargar información almacenada.

El sistema debe recuperar los datos guardados al iniciar.

- ✓ RF36. Confirmar operaciones.

El sistema debe mostrar mensajes de confirmación después de cada acción exitosa.

- ✓ RF37. Mostrar mensajes de error.

El sistema debe informar errores cuando se produzcan operaciones inválidas.

- ✓ RF38. Validar datos ingresados.

El sistema debe verificar que la información registrada sea correcta.

- ✓ RF39. Generar respaldo de datos.

El sistema debe permitir crear copias de seguridad de la información.

✓ RF40. Salir del sistema.

El sistema debe cerrar la aplicación de manera segura.

7. Historias de Usuario

Las historias de usuario nos ayudan a describir funcionalidades del sistema desde la perspectiva del operador en de la ferretería, quien es el usuario principal del sistema. Cada uno tiene una función como Rol, acción, Beneficio

HU01

Como operador de almacén quiero registrar productos para mantener actualizado el inventario.

HU02

Como operador de almacén quiero modificar productos para corregir información registrada.

HU03

Como operador de almacén quiero eliminar productos para retirar artículos que ya no se utilizan.

HU04

Como operador de almacén quiero buscar productos para encontrarlos rápidamente dentro del inventario.

HU05

Como operador de almacén quiero consultar el stock para conocer la disponibilidad de los productos.

HU06

Como operador de almacén quiero registrar entradas de mercadería para mantener actualizado el inventario.

HU07

Como operador de almacén quiero registrar salidas de productos para controlar los movimientos del almacén.

HU08

Como operador de almacén quiero que el sistema actualice automáticamente el stock para evitar errores manuales.

HU09

Como operador de almacén quiero consultar el historial de movimientos para tener trazabilidad de los productos.

HU10

Como operador de almacén quiero visualizar productos con stock crítico para realizar reposiciones oportunamente.

1. Introducción y Contextualización del Caso Ferretería

De acuerdo con las directrices de la Práctica 5 del curso de Programación Orientada a Objetos, este documento detalla la implementación y el análisis de las relaciones estructurales entre clases en Java, aplicadas a un entorno de negocio real: un Sistema de Gestión de Ferretería. Las relaciones de Agregación y Composición son fundamentales para modelar cómo los componentes individuales interactúan e integran sistemas complejos, asegurando la modularidad, la reutilización del código y el mantenimiento seguro de la arquitectura.

2. Definición de Conceptos Estructurales

2.1. Relación de Agregación

La agregación representa una relación de totalidad/parcialidad débil (un tipo especial de asociación). En el contexto de nuestra ferretería, un Cliente y una Empresa Ferretera representan una agregación. El cliente forma parte del flujo de negocio, pero si la ferretería cierra o la instancia del objeto "Ferreteria" es destruida en memoria, el objeto 'Cliente' sigue existiendo de manera independiente fuera de ella.

2.2. Relación de Composición

La composición representa una relación de ciclo de vida fuerte y estrictamente acoplada. Si el objeto contenedor (el Todo) deja de existir, las partes asociadas se destruyen de manera automática. En nuestro modelo, la relación entre la Ferretería y sus Tiendas o Almacenes es de composición. Un almacén ferretero no tiene sentido de existir en el sistema si la empresa ferretera principal es eliminada.

3. Implementación del Código Fuente (Netbeans)

Clase 1: Cliente.java (Componente Independiente de Agregación)

```
package t1.ferreteria.caso5;

public class Cliente {
    private String dni;
    private String nombre;

    public Cliente(String dni, String nombre) {
        this.dni = dni;
        this.nombre = nombre;
    }
}
```

```
}

    public String getDni() { return dni; }
    public String getNombre() { return nombre; }

    public void mostrarCliente() {
        System.out.println(" [Cliente] DNI: " + dni + " | Nombre: " + nombre);
    }
}
```

Clase 2: Almacen.java (Componente Composición)

```
package t1.ferreteria.caso5;

public class Almacen {
    private String codigo;
    private String tipoMaterial;

    public Almacen(String codigo, String tipoMaterial) {
        this.codigo = codigo;
        this.tipoMaterial = tipoMaterial;
    }

    public String getCodigo() { return codigo; }
    public String getTipoMaterial() { return tipoMaterial; }

    public void mostrarAlmacen() {
        System.out.println(" [Almacen] Código: " + codigo + " | Sector: " +
            tipoMaterial);
    }
}
```

Clase 3: Ferreteria.java (Clase Contenedora)

```
package t1.ferreteria.caso5;

import java.util.ArrayList;

public class Ferreteria {
    private String nombreEmpresa;
    private ArrayList<Cliente> listaClientes; // Relación de Agregación
    private ArrayList<Almacen> listaAlmacenes; // Relación de Composición

    public Ferreteria(String nombreEmpresa) {
        this.nombreEmpresa = nombreEmpresa;
        this.listaClientes = new ArrayList<>();
        this.listaAlmacenes = new ArrayList<>();
    }
}
```

```
// Método para agregar Cliente (Agregación: el objeto se crea fuera y se añade)
public void agregarCliente(Cliente cliente) {
    if (cliente != null) {
        this.listaClientes.add(cliente);
    }
}

// Método para agregar Almacén (Composición: la Ferretería instancia
directamente sus partes)
public void crearAlmacen(String codigo, String tipoMaterial) {
    Almacen nuevoAlmacen = new Almacen(codigo, tipoMaterial);
    this.listaAlmacenes.add(nuevoAlmacen);
}

public void mostrarReporteEstructural() {

    System.out.println("=====
    ==");
    System.out.println("EMPRESA FERRETERA: " +
    nombreEmpresa.toUpperCase());

    System.out.println("=====
    ==");

    System.out.println("SECTORES DE ALMACENAMIENTO (Composición):");
    for (Almacen a : listaAlmacenes) {
        a.mostrarAlmacen();
    }

    System.out.println("
    CARTERA DE CLIENTES ASOCIADOS (Agregación):");
    for (Cliente c : listaClientes) {
        c.mostrarCliente();
    }

    System.out.println("=====
    ==");
    }
}
```

Clase 4: MainFerreteria.java (Controlador de Ejecución)

```
package t1.ferreteria.caso5;

public class MainFerreteria {
```

```
public static void main(String[] args) {
    // Instancia principal de la empresa contenedora
    Ferreteria miFerreteria = new Ferreteria("Ferretería El Tornillo Dorado");

    // 1. Demostración de Composición (La ferretería administra el ciclo de vida)
    miFerreteria.crearAlmacen("ALM-01", "Herramientas Manuales y Eléctricas");
    miFerreteria.crearAlmacen("ALM-02", "Materiales de Construcción y
    Acabados");

    // 2. Demostración de Agregación (Los clientes existen por separado)
    Cliente c1 = new Cliente("45781293", "Juan Pérez");
    Cliente c2 = new Cliente("70129481", "María Mendoza");

    miFerreteria.agregarCliente(c1);
    miFerreteria.agregarCliente(c2);

    // Despliegue del reporte estructurado
    miFerreteria.mostrarReporteEstructural();
}
}
```

4. Análisis Técnico de Ciclo de Vida y Acoplamiento

El análisis de la arquitectura del software desarrollado revela que la implementación de colecciones lógicas mediante “ArrayList” permite flexibilizar el crecimiento del inventario y de la cartera corporativa. En la Composición, al usar el método “crearAlmacen(String, String)”, la instanciación de la clase ‘Almacen’ ocurre estrictamente dentro de la clase “Ferreteria”. Esto imposibilita el acceso o existencia autónoma del almacén fuera del contenedor principal. Por el contrario, en la Agregación, los clientes son inyectados como parámetros ya instanciados desde el exterior (“agregarCliente(Cliente)”), cumpliendo a cabalidad con las pautas de diseño limpio especificadas en las Sesiones del curso.

Introducción y Matriz de Control de los 20 Requerimientos

Se presenta la matriz detallada con la definición de 20 requerimientos de software aplicados a la empresa ferretera, organizados por tipo, descripción operativa, historia de usuario asociada y su estado actual de implementación.

ID / Tipo	Descripción Operativa	Historia	Estado Final
RF-01 (GUI)	Registrar nuevos productos	HU-01	Implementado
RF-02 (GUI)	Listar el inventario total con barras de	HU-02	Implementado

desplazamiento.

RF-03 (Lógica)	Validar que los precios y existencias sin valores negativos.	HU-01	Implementado
RF-04 (GUI)	Registrar clientes	HU-03	Implementado
RF-05 (GUI)	Visualizar la cartera de clientes	HU-04	Implementado
RF-06 (Lógica)	Validar el formato de cadena del RUC (solo 11 caracteres numéricos) al guardar.	HU-03	Implementado
RF-07 (GUI)	Crear y dar de alta sucursales internas de almacenamiento.	HU-05	Implementado
RF-08 (GUI)	Modificar los datos	HU-05	Implementado
RF-09 (Lógica)	Controlar la composición del ciclo de vida <i>eliminando almacenes si la matriz se destruye.</i>	HU-05	Implementado
RF-10 (GUI)	Registrar ventas por cliente y producto.	HU-06	Implementado
RF-11 (Lógica)	Calcular IGV (18%) y total neto de la orden ferretera.	HU-06	Implementado
RF-12 (Lógica)	Descontar de forma inmediata el stock del producto al confirmar la transacción de venta.	HU-06	Implementado
RF-13 (GUI)	Limpiar los búferes visuales de las cajas de texto.	HU-02	Implementado

RF-14 (GUI)	Cerrar la ventana.	HU-02	Implementado
RF-15 (Lógica)	Capturar excepciones numéricas.	HU-01	Implementado
RF-16 (GUI)	Mostrar cuadros emergentes (JOptionPane) para confirmar acciones exitosas o errores.	HU-01	Implementado
RF-17 (Lógica)	Inicializar las colecciones ArrayList al instanciar el constructor.	HU-05	Implementado
RF-18 (Lógica)	Impedir el registro de campos vacíos o con cadenas en blanco mediante trim().isEmpty().	HU-01	Implementado
RNF-19 (Seguridad)	encapsulamiento estricto de atributos de negocio bajo modificador private.	HU-07	Cumplido
RNF-20 (Usabilidad)	Desplegar ventanas centradas.	HU-07	Cumplido

2. Historias de Usuario y Criterios de Aceptación

Historia de Usuario 01 (HU-01): Gestión de Productos con Java Swing

****Como:**** Encargado de Almacén

****Quiero:**** Registrar productos ferreteros a través de una interfaz gráfica intuitiva

****Para:**** Mantener el catálogo actualizado sin usar comandos de consola.

****Criterios de Aceptación:****

1. El sistema debe validar que el precio no sea menor o igual a cero empleando un bloque try-catch.
2. Al registrar con éxito, se debe alertar al operador mediante un JOptionPane y limpiar automáticamente los cuadros de texto.

Historia de Usuario 02 (HU-02): Reporte de Inventario con JScrollPane

****Como:**** Administrador del Negocio

****Quiero:**** Visualizar la lista consolidada de materiales en el panel visual

****Para:**** Auditar las existencias globales de la ferretería de manera inmediata.

****Criterios de Aceptación:****

1. La interfaz debe cargar los productos en un JTextArea que no permita la edición de texto por parte del usuario.
2. Si el volumen de datos excede las dimensiones del panel, se deben activar barras de desplazamiento dinámicas.

3. Implementación del Software Final (Código Java Swing)

A continuación, se detalla el código fuente funcional unificado e integrado en Eclipse para dar cumplimiento absoluto al software de la Práctica de Campo 6.

Clase de Dominio: Producto.java

```
package pc6.ferreteria;

public class Producto {
    private String codigo;
    private String descripcion;
    private double precio;

    public Producto(String codigo, String descripcion, double precio) {
        this.codigo = codigo;
        this.descripcion = descripcion;
        this.precio = precio;
    }

    public String getCodigo() { return codigo; }
    public String getDescripcion() { return descripcion; }
    public double getPrecio() { return precio; }
}
```

Clase de Interfaz y Control de Eventos: VentanaPrincipal.java

```
package pc6.ferreteria;

import javax.swing.*;
import java.awt.event.ActionEvent;
import java.awt.event.ActionListener;
import java.util.ArrayList;

public class VentanaPrincipal extends JFrame implements ActionListener {
    private JLabel lblTitulo, lblCodigo, lblDesc, lblPrecio;
```

```
private JTextField txtCodigo, txtDesc, txtPrecio;
private JButton btnRegistrar, btnMostrar, btnLimpiar;
private JTextArea txtReporte;
private JScrollPane scrollPane;
private ArrayList<Producto> inventario;

public VentanaPrincipal() {
    inventario = new ArrayList<>();

    // Configuración de ventana (Layout Absoluto según Sesión 06)
    this.setTitle("Ferretería El Tornillo de Oro - Sistema Integrado PC6");
    this.setSize(500, 480);
    this.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
    this.setLocationRelativeTo(null);
    this.setLayout(null);

    lblTitulo = new JLabel("PANEL DE CONTROL GENERAL DE INVENTARIO");
    lblTitulo.setBounds(100, 15, 300, 25);
    this.add(lblTitulo);

    lblCodigo = new JLabel("Código de Ítem:");
    lblCodigo.setBounds(40, 60, 120, 25);
    this.add(lblCodigo);

    txtCodigo = new JTextField();
    txtCodigo.setBounds(160, 60, 280, 25);
    this.add(txtCodigo);

    lblDesc = new JLabel("Descripción:");
    lblDesc.setBounds(40, 100, 120, 25);
    this.add(lblDesc);

    txtDesc = new JTextField();
    txtDesc.setBounds(160, 100, 280, 25);
    this.add(txtDesc);

    lblPrecio = new JLabel("Precio (S/.):");
    lblPrecio.setBounds(40, 140, 120, 25);
    this.add(lblPrecio);

    txtPrecio = new JTextField();
    txtPrecio.setBounds(160, 140, 280, 25);
    this.add(txtPrecio);

    // Botonera de Operaciones
```

```
btnRegistrar = new JButton("Registrar");
btnRegistrar.setBounds(50, 190, 110, 30);
btnRegistrar.addActionListener(this);
this.add(btnRegistrar);

btnMostrar = new JButton("Mostrar Stock");
btnMostrar.setBounds(185, 190, 120, 30);
btnMostrar.addActionListener(this);
this.add(btnMostrar);

btnLimpiar = new JButton("Limpiar");
btnLimpiar.setBounds(330, 190, 110, 30);
btnLimpiar.addActionListener(this);
this.add(btnLimpiar);

// Área de Reporte con Scroll
txtReporte = new JTextArea();
txtReporte.setEditable(false);
scrollPane = new JScrollPane(txtReporte);
scrollPane.setBounds(40, 245, 400, 160);
this.add(scrollPane);
}

@Override
public void actionPerformed(ActionEvent e) {
    if (e.getSource() == btnRegistrar) {
        try {
            String cod = txtCodigo.getText().trim();
            String desc = txtDesc.getText().trim();
            String preStr = txtPrecio.getText().trim();

            if (cod.isEmpty() || desc.isEmpty() || preStr.isEmpty()) {
                JOptionPane.showMessageDialog(this, "Error: Debe completar todos
los campos.", "Campos Vacíos", JOptionPane.WARNING_MESSAGE);
                return;
            }

            double precio = Double.parseDouble(preStr);
            if (precio <= 0) {
                JOptionPane.showMessageDialog(this, "El precio debe ser un número
positivo.", "Validación de Datos", JOptionPane.ERROR_MESSAGE);
                return;
            }

            inventario.add(new Producto(cod, desc, precio));
```

```
JOptionPane.showMessageDialog(this, "Producto añadido con éxito al
inventario corporativo.", "Éxito", JOptionPane.INFORMATION_MESSAGE);
limpiarCampos();
```

```
    } catch (NumberFormatException ex) {
        JOptionPane.showMessageDialog(this, "Error: El formato de precio
introducido no es válido.", "Excepción de Formato",
JOptionPane.ERROR_MESSAGE);
    }
```

```
    } else if (e.getSource() == btnMostrar) {
        if (inventario.isEmpty()) {
            txtReporte.setText("El inventario ferretero se encuentra vacío
actualmente.");
            return;
        }
    }
```

```
txtReporte.setText("=====
===
");
    txtReporte.append("    REPORTE DE EXISTENCIAS - MATRIZ PC6
");
```

```
txtReporte.append("=====
===
");
    for (Producto p : inventario) {
        txtReporte.append(" Cód: " + p.getCodigo() + " | " + p.getDescripcion() + "
→ S/. " + p.getPrecio() + "
");
    }
```

```
txtReporte.append("=====
===
");
    } else if (e.getSource() == btnLimpiar) {
        limpiarCampos();
        txtReporte.setText("");
    }
}
```

```
private void limpiarCampos() {
    txtCodigo.setText("");
    txtDesc.setText("");
    txtPrecio.setText("");
    txtCodigo.requestFocus();
}
```

```
}  
}
```

Clase Controladora de Entrada: MainApp.java

```
package pc6.ferreteria;  
  
public class MainApp {  
    public static void main(String[] args) {  
        // Inicialización y despliegue del software en el Event Dispatch Thread  
        VentanaPrincipal panelVisual = new VentanaPrincipal();  
        panelVisual.setVisible(true);  
    }  
}
```

4. Análisis de Resultados y Conclusión del Proyecto

la Práctica de Campo 6 evidencia el diseño estructurado bajo las reglas lógicas de la POO permite soportar matrices complejas de requerimientos sin perder estabilidad. Al mapear y validar 20 especificaciones precisas, demuestra que la separación de responsabilidades (Clase Entidad y Clase Ventana) mitiga el acoplamiento crítico de código.

La implementación visual utilizando coordenadas absolutas (null layout) y escuchadores de acciones proporciona al operador un sistema con tolerancia a fallos mediante el aislamiento de excepciones de formato, alcanzando los niveles óptimos de calidad, legibilidad y completitud exigidos por los estándares universitarios actuales.

Pendiente de mejoras futuras e implementación real.

Bibliografía:

Barreto, L., Guillen, M., Gómez, J., & Damián, E. (2022). *Sistema de gestión de ventas e inventario* [Trabajo de curso]. Universidad Tecnológica del Perú.

Deitel, P. J., & Deitel, H. M. (2012). *Java: Cómo programar* (9.ª ed.). Pearson Educación.

Instituto Politécnico Nacional. (2010). *Sistema de inventario orientado a objetos en Java* [Tesis]. CIATEQ. <https://tesis.ipn.mx/bitstream/handle/123456789/5757/C7.1410.pdf>

Larman, C. (2003). *UML y patrones: Una introducción al análisis y diseño orientado a objetos*. Prentice Hall.

Laudon, K. C., & Laudon, J. P. (2016). *Sistemas de información gerencial* (14.ª ed.). Pearson.

Román Arce, J. (2023). *Implementación de un sistema de gestión de inventarios* [Tesis de pregrado, Universidad Continental].

https://repositorio.continental.edu.pe/bitstream/20.500.12394/13413/2/IV_FIN_108_TE_Roman_Arce_2023.pdf